

Meta-Population extensions

Spencer J. Fox Qianying (Ruby) Lin Jesse Wheeler

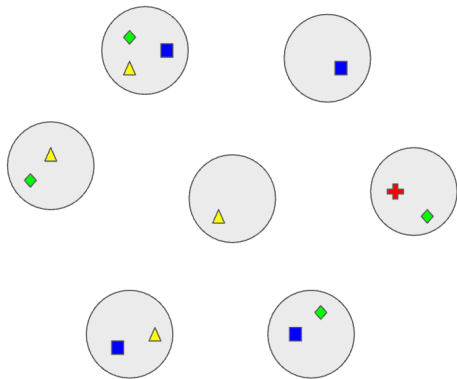
Package Extensions

Mathematically, POMP models are very versatile, and many of the ideas useful to the models described so far are also applicable in more general settings. This gives rise to a few useful extensions of the `pomp` package, which we try to describe here.

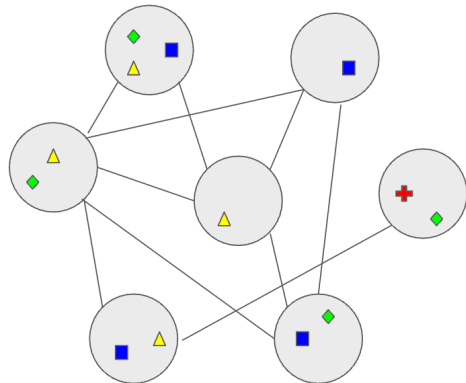
- ▶ Meta-population data and models
 - ▶ `panelPomp`
 - ▶ `spatPomp`
- ▶ Genomic / phylogenetic data
 - ▶ `phylopomp`

Each of these packages extend the `pomp` package to other useful cases.

Meta-population: Data and Models



panelPomp: data from related,
independent systems.



spatPomp: data from a single
interacting system.

Meta-population packages: Examples

Both packages require `pomp` to be installed, and the internal workings of both packages actually use many of the `pomp` functions. Thus, familiarity with `pomp` is a prerequisite.

- ▶ `panelPomp`:
 - ▶ We have *panel* or *longitudinal* data.
 - ▶ Several independent yet related systems. Information from all systems is useful, not just one location.
 - ▶ Simultaneously allows for features shared by each system as well as system unique features.
 - ▶ Example: endemic COVID-19 measured in 3 locations: New York, London, Atlanta.
- ▶ `spatPomp`:
 - ▶ The name comes from the idea that we have spatially explicit POMP models.
 - ▶ The data at multiple locations are thought of as location-specific measurements from the same system. That is, the dynamic systems underlying the data are connected.
 - ▶ Example: Modeling early stages of COVID-19 in China. Here, we might have measurements for many cities, but at first cases were only found in Wuhan.

PanelPomp: a collection of POMP models

A panelPomp model is really just a collection of pomp models. This is also how they are built:

```
library(panelPomp)

mod1 <- pomp(..., params = c('p1' = 0.1, 'p2' = 1.2, 'p3' = 0.9))
mod2 <- pomp(..., params = c('p1' = 0.1, 'p2' = 1.1, 'p3' = 0.6))
mod3 <- pomp(..., params = c('p1' = 0.1, 'p2' = 1.5, 'p3' = 0.75))

ppomp <- panelPomp(
  object = list(mod1, mod2, mod3),
  shared = c("p1" = 0.1), # Shared-value parameters
  specific = c("p2", "p3") # Unit-Specific parameters
)
```

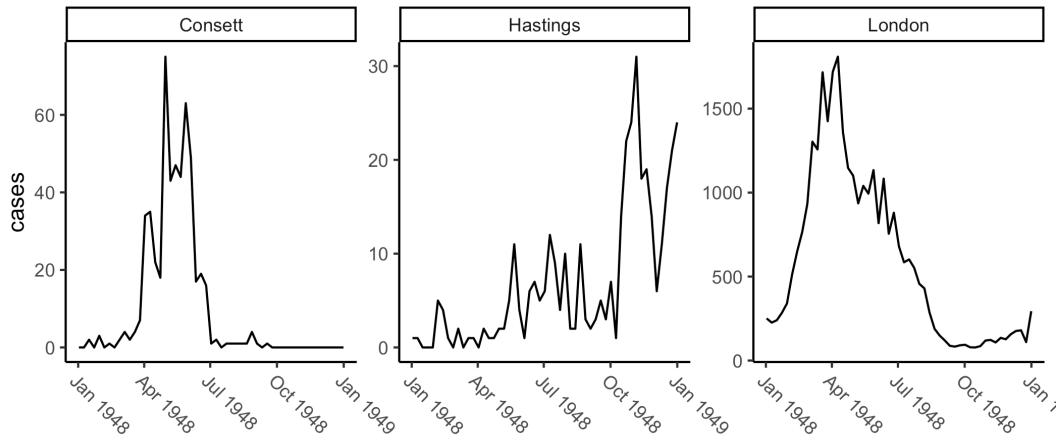
Measles Example

We can build off of the measles example by looking at UK measles from multiple cities, and building an SEIR model for the data. There is built in models and data in `panelPomp`

```
measSIR <- panelMeasles(  
  units = c("Consett", "London", "Hastings"),  
  first_year = 1948,  
  last_year = 1948  
)
```

Measles Example: Figure

```
plot(measSIR)
```



panelPomp: parameters

Parameter names in panelPomp have the following conventions. If the parameter is shared, it just is called by the name. If it is unit-specific, the name of the unit follows the name of the parameter: <param>[<unit>].

`coef(measSIR)`

mu	sigma[Consett]	gamma[Consett]
2.00e-02	4.26e+01	1.72e+02
rho[Consett]	R0[Consett]	amplitude[Consett]
6.50e-01	3.59e+01	2.00e-01
alpha[Consett]	iota[Consett]	cohort[Consett]
1.01e+00	7.31e-02	3.10e-01
psi[Consett]	S_0[Consett]	E_0[Consett]
4.06e-01	3.22e-02	1.83e-05
I_0[Consett]	R_0[Consett]	sigmaSE[Consett]
1.97e-05	9.68e-01	7.12e-02
sigma[Hastings]	gamma[Hastings]	rho[Hastings]
5.63e+01	7.41e+01	6.95e-01

panelPomp: shared vs unit-specific

A key feature of `panelPomp` objects is which parameters are shared, which are unit-specific. If parameters are shared, that means they have the same value for all units. We can inspect and modify which parameters are which using the functions `shared` and `specific`:

```
shared(measSIR)
```

```
mu  
0.02
```

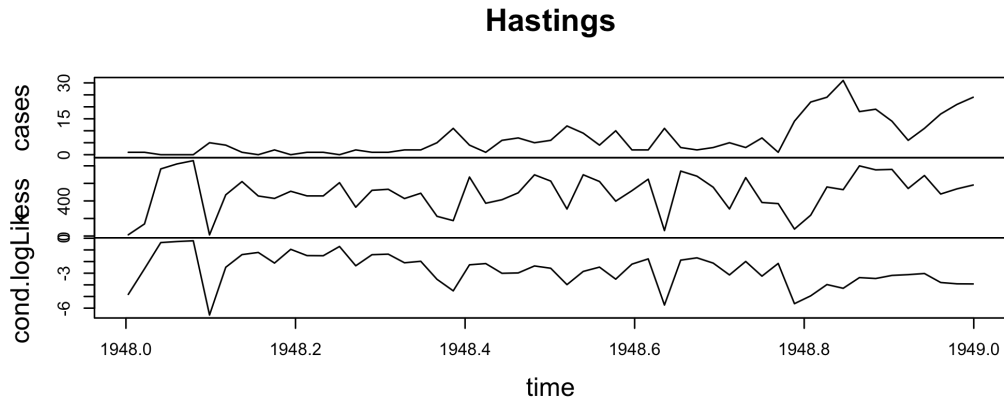
```
shared(measSIR) <- c(shared(measSIR), 'alpha' = 1)  
shared(measSIR)
```

```
mu alpha  
0.02  1.00
```

panelPomp Functions

panelPomp is the easiest extension: If you can build one pomp, you can build multiple into a single panelPomp object. Existing functions and algorithms are similar as well!

```
pfilter(measSIR, Np = 1000) |> plot(unit = 'Hastings')
```



panelPomp: iterated filtering

We need a new algorithm for conducting maximum likelihood, either the panel iterated filter (PIF), or marginalized panel iterated filter (MPIF).

We don't even need to change the code!

```
mif2Out <- mif2(  
  measSIR, rw.sd = rw_sd(rho = 0.02, R_0 = 0.02),  
  Nmif = 10, Np = 200, cooling.fraction.50 = 0.5,  
  block = TRUE # block = TRUE does MPIF, will become default.  
)
```

Capable of fitting more than 2500 time series simultaneously (Wheeler, Abkemeier, and Ionides 2025).

panelPomp: likelihood I

Because units are independent, the likelihood can be computed independently for each unit, and the total log-likelihood is the sum of unit log-likelihoods.

```
unitLogLik(mif2Out)
```

```
   Consett  Hastings    London  
-111.8653 -139.0361 -499.9111
```

```
sum(unitLogLik(mif2Out))
```

```
[1] -750.8125
```

```
logLik(mif2Out)
```

```
[1] -750.8125
```

panelPomp: likelihood II

When we do repeated particle filters, we get better estimates if we first average log-likelihood over the units, then take the sum.

```
library(foreach)
library(doParallel)
library(doRNG)
registerDoParallel(detectCores()-1)
foreach(rep=1:10, .combine = c) %dopar% {
  pfilter(measSIR, Np = 5000)
} -> pfOut # Object of length 10
```

panelPomp: likelihood III

Like in `pomp`, we can average results using `logmeanexp`. However, because there is more than one way to do it, we have a new function `panel_logmeanexp`. The preferred approach is averaging over the unit, then summing:

```
panel_logmeanexp(  
  sapply(pfOut, unitLogLik), MARGIN = 1, se = TRUE  
)
```

```
              se  
-698.304264    2.037088
```

We usually have larger error than in `pomp`, because there is error for each unit that adds up across the entire `panelPomp` object.

panelPomp: summary

- ▶ Easy extension of `pomp` that allows to fit related models to many locations.
- ▶ Useful for (approximately) independent systems (e.g., outbreaks within various hospitals, approximately geographically isolated outbreaks of same disease, controlled randomized experiments,...)
- ▶ Very useful if limited data is available, we can pool information from all sources.
- ▶ Allows for estimating shared and unit-specific parameters.
- ▶ To learn more, see the panelPomp research paper (to appear in the R Journal) (Bretó et al. 2025), or the package website <https://panelpomp-org.github.io/>

panelPomp: Exercise

Install the `panelPomp` package from CRAN via `install.packages("panelPomp")`. Create one of the prebuilt `panelPomp` models using a constructor function: `panelGompertz()`, `panelMeasles()`, `contacts`, or `panelRandomWalk()`. Our goal is to build some familiarity with `panelPomp` models.

Check the documentation to learn more about the model (e.g., `?panelGompertz`). How many units does it have by default? How many observations and parameters? Which parameters are shared and which unit specific? Can you change the parameter type? What happens if you `plot` the object?

Once you feel like you understand the object, try running a particle filter and iterated filter (PIF or MPIF, implemented via `mif2`).

spatPomp: Introduction

spatPomp is another extension that can be used to fit meta-population models. The difference between spatPomp and panelPomp is that spatPomp allows for dependence between units.

- ▶ Both mathematically and computationally, this is a much more difficult task.
- ▶ It can lead to very interesting and high-dimensional models.

Note that any panelPomp model can be expressed as a spatPomp, so you can use spatPomp methods to fit panelPomp models. However, if a panelPomp is appropriate, it is both theoretically and computationally more efficient to treat the model as a panelPomp.

High-dimensional models

spatPomp is relatively new, the software paper was published last year (Asfaw et al. 2024b). It enables fitting higher-dimensional POMP models than has been possible in the past.

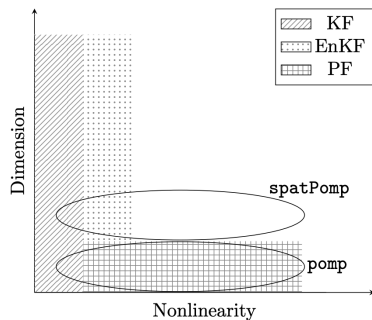


Figure 1: Limits of POMP modeling. Credit: Asfaw et al. 2024.

Dependent States

Unlike panelPomp, the latent process $X(t)$ is assumed to be linked for each unit (e.g., infectious individuals from city 1 can infect susceptible individuals in city 2).

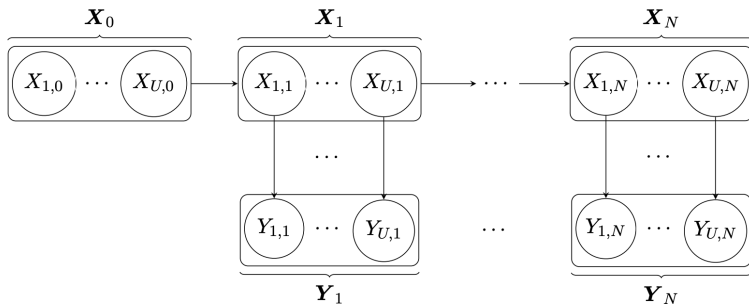


Figure 2: Credit: Asfaw et al. 2024.

Implications

- ▶ Allowing for spatially dependent latent processes can lead to more interesting, more realistic models.
- ▶ However, they become more complicated to build in practice, because we have to carefully describe how the systems interact.
- ▶ Mathematically, these models are also more complex. Until very recently, plug-and-play methods for fitting and evaluating these models did not exist.

spatPomp: methods

Parts of a spatPomp that can be implemented

Method	Argument to spatPomp()	Mathematical terminology
dunit_measure	dunit_measure	Evaluate $f_{Y_{u,n} X_{u,n}}(y_{u,n} x_{u,n}; \theta)$
runit_measure	runit_measure	Simulate from $f_{Y_{u,n} X_{u,n}}(y_{u,n} x_{u,n}; \theta)$
eunit_measure	eunit_measure	Evaluate $e_{u,n}(x, \theta) = E[Y_{u,n} X_{u,n} = x; \theta]$
vunit_measure	vunit_measure	Evaluate $v_{u,n}(x, \theta) = \text{Var}[Y_{u,n} X_{u,n} = x; \theta]$
munit_measure	munit_measure	$m_{u,n}(x, V, \theta) = \psi$ if $v_{u,n}(x, \psi) = V$, $e_{u,n}(x, \psi) = e_{u,n}(x, \theta)$
rprocess	rprocess	Simulate from $f_{X_n X_{n-1}}(\mathbf{x}_n \mathbf{x}_{n-1}; \theta)$
dprocess	dprocess	Evaluate $f_{X_n X_{n-1}}(\mathbf{x}_n \mathbf{x}_{n-1}; \theta)$
rmeasure	rmeasure	Simulate from $f_{Y_n X_n}(\mathbf{y}_n \mathbf{x}_n; \theta)$
dmeasure	dmeasure	Evaluate $f_{Y_n X_n}(\mathbf{y}_n \mathbf{x}_n; \theta)$
rprior	rprior	Simulate from the prior distribution $\pi(\theta)$
dprior	dprior	Evaluate the prior density $\pi(\theta)$
rinit	rinit	Simulate from $f_{X_0}(\mathbf{x}_0; \theta)$
timezero	t0	t_0
time	times	$t_{1:N}$
obs	data	$\mathbf{y}_{1:N}^*$
states	—	$\mathbf{x}_{0:N}$
coef	params	θ

Model specification (rinit)

```
measles_rinit <- spatPomp_Csnippet(  
  unit_statenames = c('S','E','I','C'),  
  unit_ivpnames = c('S','E','I'),  
  unit_covarnames = c('P'),  
  code = "  
  for (int u=0; u<U; u++) {  
    S[u] = round(P[u]*S_0[u]);  
    E[u] = round(P[u]*E_0[u]);  
    I[u] = round(P[u]*I_0[u]);  
    C[u] = 0;  
  }  
"  
)
```

Model specification (rprocess)

```
measles_rprocess <- spatPomp_Csnippet(  
  unit_statenames = c('S','E','I','C'),  
  code = "double beta, seas, Ifrac, mu[7], dN[7];  
  int u, v;  
  int BS=0, SE=1, SD=2, EI=3, ED=4, IR=5, ID=6;  
  ...  
  for (u = 0 ; u < U ; u++) {  
    ...  
  
    S[u] += dN[BS] - dN[SE] - dN[SD];  
    E[u] += dN[SE] - dN[EI] - dN[ED];  
    I[u] += dN[EI] - dN[IR] - dN[ID];  
    C[u] += dN[EI];  
  }"  
)
```

Model specification (dmeasure)

```
measles_dunit_measure <- spatPomp_Csnippet("  
  double m = rho*C;  
  double v = m*(1.0-rho+psi*psi*m);  
  lik = dnorm(cases,m,sqrt(v),give_log);"  
)
```


Model specification

```
measles_spatPomp <- spatPomp(  
  data = ...,  
  unit_statenames = c('S','E','I','R','C'),  
  unit_accumvars = c('C'),  
  paramnames = ...,  
  rinit = measles_rinit,  
  rprocess = euler(measles_rprocess, delta.t=1/365),  
  dunit_measure = measles_dunit_measure,  
  eunit_measure = measles_eunit_measure,  
  vunit_measure = measles_vunit_measure,  
  runit_measure = measles_runit_measure,  
  dmeasure = measles_dmeasure,  
  rmeasure = measles_rmeasure  
)
```

Block particle filter (bpfilter)

To perform inference, it is similar to `pomp`, but we will use an algorithm called the Block-Particle Filter (BPF). Particle filters don't work well in high-dimensions (even as low as 3 dimensions is “high”). The BPF is similar to the particle filter, but does the following:

- ▶ Simulate the entire process forward with `rprocess`.
- ▶ Use `dunit_meas` to calculate weights independently for all units, and update particles independently.

Pro: This algorithm is surprisingly fast and effective. Best if coupling between units is weak.

Con: Does not satisfy conservation of mass properties. Alternative options can be found in the `spatPomp` tutorial (Asfaw et al. 2024a).

Iterated block particle filter (IBPF)

Like iterating filtering is a special type of particle filter, with model parameters doing a random walk, we can iterate the BPF to get an IBPF to fit model parameters (Ionides, Ning, and Wheeler 2024).

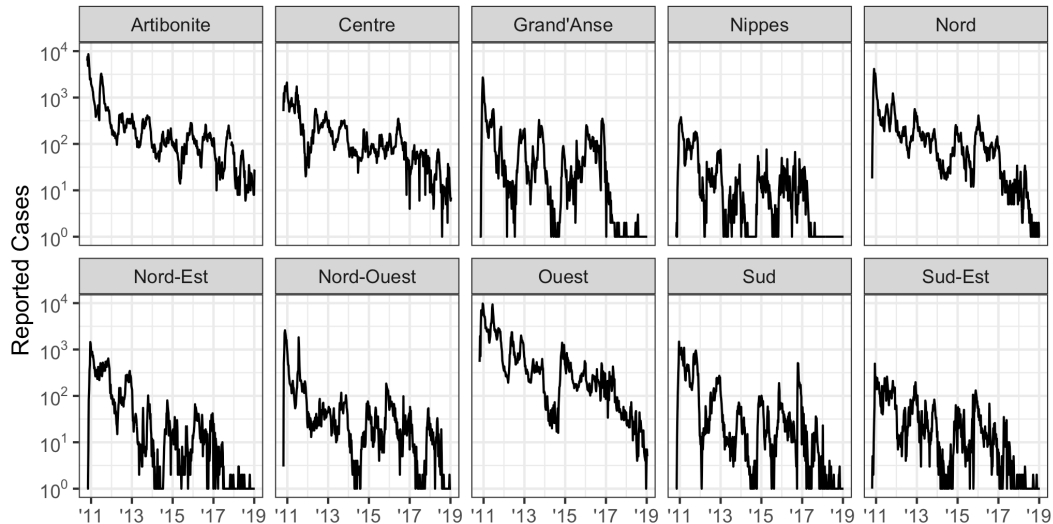
Like the BPF, the IBPF is surprisingly effective at maximizing the likelihood (Asfaw et al. 2024a), but does not conserve mass.

Example: Cholera in Haiti

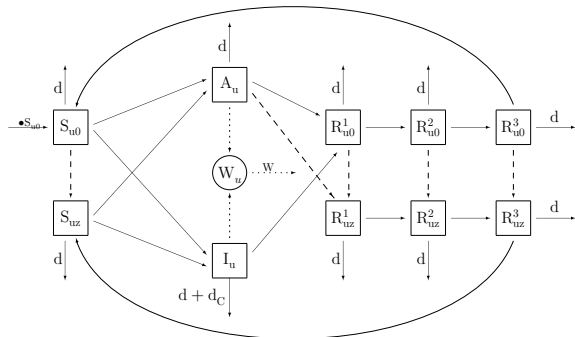


- ▶ Haiti experienced a cholera outbreak following the devastating 2010 earthquake.
- ▶ From 2010-2019, more than 800,000 recorded cases, making it one of the largest recorded outbreaks.
- ▶ Oral cholera vaccination (OCV) is available, but in limited supply.
- ▶ A modeling goal is to study the potential impact of various vaccination strategies.
- ▶ Image credit: UNICEF.
- ▶ This example is from Wheeler, et al. (2024).

Haiti Data



Cholera Model



- ▶ SIAR3W Model
- ▶ Spatial Dependence between units.
- ▶ Stochastic transmission rates.
- ▶ Overdispersed Markov counting system.
- ▶ Rainfall driven transmission.
- ▶ Environmental reservoir of bacteria.
- ▶ Adjustments for Hurricane Mathew (Oct 2016).
- ▶ For details on implementation, see Wheeler, et al. (2024).

Pre-built model in the `haitipkg` R package

Rather than build this from scratch, we can use a constructor function from `haitipkg`.

```
devtools::install_github('jeswheel/haitipkg')  
sp_mod <- haitipkg::haiti3_spatPomp()
```

Performing inference for this model is similar to the `pomp` Measles example, but it takes much longer to compute in practice

```
bpf_out <- bpfilter(sp_mod, Np = 1000, block_size = 1)
```

This took 41.16 seconds to run.

Parameter Estimation

The functionality for estimating parameters using IBPF is similar to using `mif2` in `pomp`:

```
## NOT RUN
ibpf(
  h3_spat,
  Nbpf = 100, # Equivalent to Nmif
  Np = 2000,
  sharedParNames = <character vector of shared parameter names>,
  unitParNames = <character vector of unit-specific parameter names>,
  spat_regression = 0.1,
  rw.sd = ...,
  cooling.fraction.50 = ...,
  block_size = 1
)
```


Confidence Intervals

For `spatPomp`, we can compute confidence intervals in a similar way as `pomp`. That is, we can fix the parameter of interest at a range of values, and maximize the likelihood for the remaining parameters.

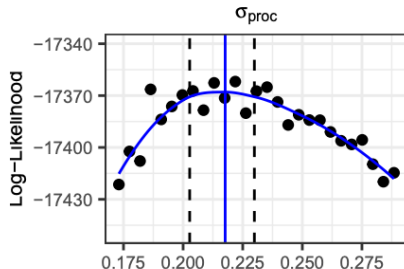


Figure 3: Confidence interval for one of the model parameters. Credit: Wheeler, et al. (2024).

spatPomp: Summary

- ▶ The spatPomp package is an extension of pomp that enables building and fitting interacting dynamic meta-population models.
- ▶ Algorithms and software are a little bit different than pomp, but similar enough that your pomp skills are applicable.
- ▶ The nature of these models means that things run much slower, for two reasons:
 - ▶ If you have data from U units, then the code is roughly U times slower for the same number of iterations and particles.
 - ▶ Because there are U times as many parameters, the likelihood surface is usually more complex, and we may need more iterations and particles than for low-dimensional systems.
- ▶ To learn more, see the package tutorial (Asfaw et al. 2024a) or the Haiti Cholera example we discussed (Wheeler et al. 2024).

References I

- Asfaw, Kidus, Joonha Park, Aaron A. King, and Edward L. Ionides. 2024a. “A Tutorial on Spatiotemporal Partially Observed Markov Process Models via the R Package spatPomp.” <https://arxiv.org/abs/2101.01157>.
- Asfaw, Kidus, Joonha Park, Aaron A. King, and Edward L. Ionides. 2024b. “spatPomp: An R Package for Spatiotemporal Partially Observed Markov Process Models.” *Journal of Open Source Software* 9 (104): 7008.
- Bretó, Carles, Jesse Wheeler, Aaron A. King, and Edward L. Ionides. 2025. “panelPomp: Analysis of Panel Data via Partially Observed Markov Processes in r.” *The R Journal* 17: 180–99. <https://doi.org/10.32614/RJ-2025-009>.
- Ionides, Edward L., Ning Ning, and Jesse Wheeler. 2024. “An Iterated Block Particle Filter for Inference on Coupled Dynamic Systems with Shared and Unit-Specific Parameters.” *Statistica Sinica* 34: 1241–62.

References II

- Wheeler, Jesse, Aaron J. Abkemeier, and Edward L. Ionides. 2025. “Iterating Marginalized Bayes Maps for Likelihood Maximization with Application to Nonlinear Panel Models.” <https://arxiv.org/abs/2511.17438>.
- Wheeler, Jesse, AnnaElaine Rosengart, Zhuoxun Jiang, Kevin Tan, Noah Treutle, and Edward L. Ionides. 2024. “Informing Policy via Dynamic Models: Cholera in Haiti.” *PLOS Computational Biology* 20 (4): 1–31. <https://doi.org/10.1371/journal.pcbi.1012032>.

License, acknowledgments, and links

- ▶ This lesson is prepared for the Simulation-based Inference for Epidemiological Dynamics module at the Summer Institute in Statistics and Modeling in Infectious Diseases, SISIMID.
- ▶ The materials build on previous versions of this course and related courses.
- ▶ Licensed under the Creative Commons Attribution-NonCommercial license. Please share and remix non-commercially, mentioning its origin. 
- ▶ Produced with R version 4.5.1 and pomp version 6.4.
- ▶ Compiled on 2025-07-23.

[Back to Lesson](#)

[R code for this lesson](#)